SECURETECH SOLUTIONS | SECURITY OPERATIONS

# INCIDENT RESPONSE REPORT

Suspected malware infection and unusual network activity on a Windows workstation

| | | | |
|---|---|---|---|
| **Incident ID** | SEC-IR-2026-001 | **Classification** | CONFIDENTIAL: internal use only |
| **Affected asset** | DESKTOP-GSQMBAK (Windows, version 22H2) | **Incident type** | Suspected malware infection; unusual network activity |
| **Investigation period** | 14 to 16 October 2026 | **Report date / version** | 16 October 2026 / v1.0 (final) |
| **Severity (final)** | **Low** (initially treated as Medium until evidence was reviewed) | **Status** | **Closed: no compromise confirmed**; monitoring and control improvements recommended |
| **Prepared by** | Senamile N Maphoso | **Prepared for** | IT Management, SecureTech Solutions |

## 1. Executive summary

The IT department reported unusual network activity and a possible malware infection on workstation DESKTOP-GSQMBAK, alongside a user report of a suspicious email attachment and an antivirus warning. Management instructed the security team to investigate, gather digital evidence, identify risks and recommend protective measures. This report records that investigation. It answers three questions: **did the antivirus work**, **is there any sign of compromise that an antivirus alert alone would not reveal**, and **what should the organisation do when a similar alert occurs**.

### Key results

**Detection worked.** Defender detected the EICAR test file as Virus:DOS/EICAR_Test_File (severity Severe) at 12:04 PM on 14 October and removed it with no user action. Merely viewing the file in Edge at 11:54 AM raised no alert.

**No evidence of an attack.** The Wireshark capture (26,316 packets) showed only the traffic generated by the test commands plus normal Microsoft background connections. Across the whole Security log, none of the 547 failed logons were remote, came from an IP address, or targeted another account, and no remote logon succeeded.

**Open concerns.** The high volume and regular pattern of failed logons caused by Microsoft Edge is unexplained, and the Security log is overwriting old events. Both are listed in the recommended controls (Section 7).

### Summary of findings

| ID | Finding | Rating | Detail |
|---|---|---|---|
| F-01 | Defender detected and removed the EICAR test file within the minute it was saved | **Positive** | Section 5.2 |
| F-02 | No alert while the file was only viewed in the browser; detection occurs on write to disk | **Low** | Section 5.2 |
| F-03 | Alert was seen only locally; the pop-up was missed and must be recovered from the notification centre | **Medium** | Section 5.2 |
| F-04 | 547 failed logons caused by Microsoft Edge, recurring about every four hours; cause not confirmed | **Low** | Section 5.4 |
| F-05 | Security log is overwriting old events (total changed between 24,819 and 24,905) | **Medium** | Section 5.4 |
| F-06 | Private firewall profile active on Day 1; Day 2 network was shared with other devices | **Medium** (to confirm) | Sections 5.1, 5.3 |
| F-07 | No unexpected network destinations, no remote or guessing-pattern logons, no remote logon succeeded | **Informational** | Sections 5.3 to 5.5 |

*Ratings are the analyst's judgement of risk to the organisation if left unaddressed. Positive means a control worked; Informational means no action is needed.*

## 2. Incident background and scope

**Trigger.** The IT department reported unusual network activity and a possible malware infection on the workstation. The user reported an unexpected email attachment and, when the file was saved, a Windows Security "Threats found" notification. It

was unclear whether the file had run and whether data was at risk. **Management's instruction** was to investigate, collect evidence, identify security risks and recommend controls.

**Approach.** To investigate safely, the suspicious attachment was reproduced with the **EICAR test file**, a 68-byte text string agreed by the antivirus industry so that every scanner reacts to it as if it were malware, even though it contains no harmful code. It matters because it lets the organisation prove that detection and response work without handling genuine malware. This was a simulation exercise using free tools, and no real business data was involved. The file was downloaded from eicar.org through the browser rather than received by email. The delivery route differs, but the endpoint reaction under test (what happens when a flagged file is written to disk) is the same.

**Scope and safety.** Protection was never disabled, no genuine malware was used, and all work was done on authorised lab equipment. The Wi-Fi network was shared, so the capture was filtered to the workstation's own address (172.20.7.223) before submission. This removed other people's traffic, which is a privacy matter as well as a clarity one.

### Event timeline

| Date | Time | Event (source) |
|---|---|---|
| 14 Oct 2026 | 11:15 AM | Baseline quick scan: 31,415 files in 47 seconds, 0 threats (Windows Security) |
| 14 Oct 2026 | 11:54 AM | EICAR file opened inline in Microsoft Edge: displayed as plain text, no alert |
| 14 Oct 2026 | 12:04 PM | File saved to Downloads: Defender raised "Threats found", logged Virus:DOS/EICAR_Test_File, action Removed |
| 16 Oct 2026 | 09:59:01 | Wireshark capture started on Wi-Fi (first packet) |
| 16 Oct 2026 | 10:11:00 | Event 4625 (failed logon, local, msedge.exe): the only 4625 inside the capture window |
| 16 Oct 2026 | 10:11:37 / 10:11:51 | ping 1.1.1.1 (ICMP) and nslookup example.com (DNS) run |
| 16 Oct 2026 | ~10:16:54 | Capture stopped (about 18 minutes in total) |

## 3. Tools used

| Tool | What it is | Why it matters here |
|---|---|---|
| **Microsoft Defender Antivirus** (Windows Security) | Built-in antivirus. **Real-time protection** scans files as they are created or opened. **Cloud-delivered protection** checks unknown files against Microsoft's online intelligence. **Automatic sample submission** sends suspicious files to Microsoft for analysis. | First line of defence and the source of the alert. **Protection history** is the audit trail: it records what was found, when, where it came from and what Defender did. |
| **Windows Defender Firewall** | Filters network connections using three profiles (Domain, Private, Public). Each profile applies different rules depending on how much the current network is trusted. | Limits what can reach the machine. The active profile decides how open the machine is to other devices on the network. |
| **Wireshark** with **Npcap**; ipconfig, ping, nslookup | Wireshark is a packet analyser; Npcap is the capture driver that lets it read raw traffic from the network adapter. **Display filters** (for example dns) narrow the view. | Shows what the computer actually communicated with; the commands produced traffic I already knew, giving a baseline. Malware often looks up and connects to remote servers, so traffic is independent evidence that does not rely on the antivirus. |
| **Event Viewer** (Security log) | Windows audit log. Event **4624** is a successful logon and **4625** a failed logon. Custom XML filters search every event, not just a sample. | Shows who or what tried to sign in, how, and from where. Password guessing and remote access leave distinctive marks here. |

## 4. Investigation methodology

The work follows the stages of the widely used incident-handling lifecycle (for example NIST SP 800-61): preparation, detection and analysis, containment, eradication, recovery and post-incident review. Days 1 and 2 cover **detection and analysis**; Section 6 sets out the remaining stages. The principle behind the order of work was to **establish what normal looks like before judging what is abnormal**:

**(1) Baseline the endpoint** (Day 1A): a clean scan and confirmed settings make any later detection meaningful. **(2) Run a controlled test** (Day 1B): trigger a known detection and record what the tools show. **(3) Collect network evidence** (Day 2A): run known commands, capture them and compare with the output. **(4) Collect host evidence** (Day 2B): examine logon events in detail, then search the whole log for attack patterns so the conclusion does not rest on three hand-picked events. **(5) Correlate and conclude** (Day 2C): compare both sources and state what they do and do not prove.

# 5. Findings

## 5.1 Workstation protection baseline (Day 1, Activity A)

| Check | Result | What it means and why it matters |
|---|---|---|
| Defender Antivirus | Active | A working scanner is the precondition for every other finding in this report. |
| Real-time protection | On | Files are scanned at the moment they are written or opened. Without it, malware is only found at the next scheduled scan, which may be hours later. |
| Cloud-delivered protection and sample submission | On / On | Lets Defender recognise brand-new threats that are not yet in its local signature database, which matters for fresh phishing attachments. |
| Firewall (Domain, Private, Public) | All On; Private active | Private is treated as a trusted network and is more permissive. This is only appropriate if the network really is trusted (see Section 7). |
| Quick scan, 11:15 AM | 0 threats; 31,415 files; 47 s | A clean baseline. The later detection can therefore be attributed to the EICAR file and not to something already present. |

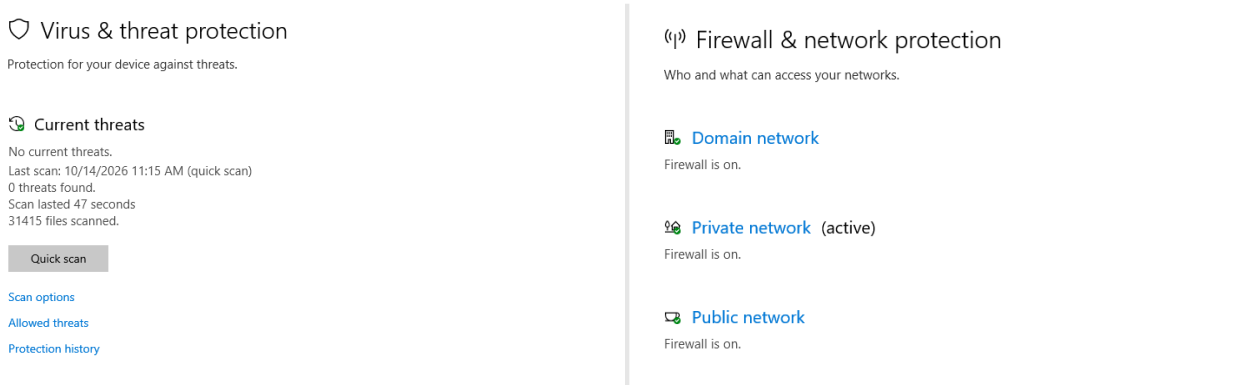



*Figure 1. (a) Quick scan result: 0 threats, 31,415 files, 47 seconds. (b) Firewall enabled for all three profiles; Private is active.*

## 5.2 Simulated malware detection (Day 1, Activity B)

Edge displayed the EICAR text at 11:54 AM without any warning. At about 12:04 PM, when the file was saved to the Downloads folder, Defender detected and removed it and a “Threats found” notification appeared. I missed the pop-up and recovered it from the Notification centre at 12:07 PM, which shows why alerts must also be logged centrally instead of relying on the user seeing a banner (Section 7).

The Protection history entry reads: **alert** Virus:DOS/EICAR_Test_File; **detected** 10/14/2026 12:04 PM; **severity** Severe; **status** Removed; **file** C:\Users\<user>\Downloads\eicar.com.txt; **source** https://secure.eicar.org/eicar.com.txt. Microsoft detection names follow a Type:Platform/Family pattern. Here “Virus” is the category, “DOS” is the platform (EICAR was originally a DOS program) and “EICAR_Test_File” is the family, so the name tells an analyst immediately what kind of threat it was. The entry also records the source URL and the process that wrote the file, which is how an analyst traces where a file came from.

**Why viewing did not alert but saving did:** antivirus intervenes when a file is written to disk or executed, not when text is merely rendered in a browser tab. For staff this means that “nothing happened when I opened it” is not proof a file is safe. For the organisation it shows that detection depends on the file reaching disk, so email filtering in front of the endpoint adds a second chance to stop an attachment (Section 7). This was a single controlled test: it confirms the control works but does not prove it would catch every threat.

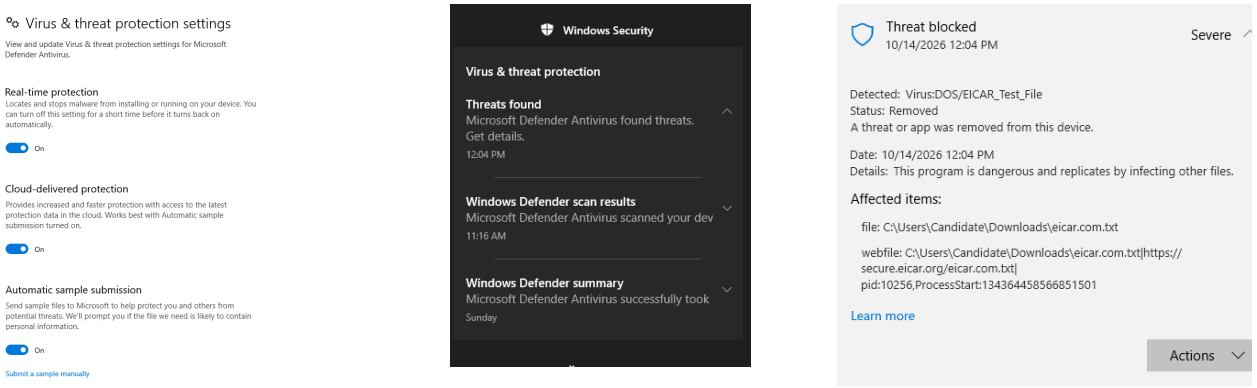



*Figure 2. (a) Real-time, cloud-delivered protection and sample submission enabled. (b) “Threats found” notification. (c) Protection history detail: Severe, Removed, with file path and source URL.*

## 5.3 Network traffic analysis (Day 2, Activity A)

Wireshark recorded 26,316 packets on the Wi-Fi interface (the only adapter showing activity). Filtering on ip.addr == 172.20.7.223 left 14,323 packets of my own traffic, which became the submission file; the unfiltered capture stays local because it contains other devices' traffic. Each protocol below is explained from what was actually seen.

| Filter | Packets | Packet examined | Source → destination (port) | What it shows |
|---|---|---|---|---|
| **icmp** | 8 | Frames 11047–11055; first request 10:11:37.636 | 172.20.7.223 → 1.1.1.1 (no port) | 4 echo requests, 4 replies; matches ping output |
| **dns** | 279 | Frame 11138, query 0x0002 A example.com, 10:11:51.475 | 172.20.7.223:51747 → 172.20.6.1:53 (UDP) | Answer in frame 11139: 172.66.147.243 and 104.20.23.154 |
| **tcp** | 6,444 | Frames 40, 44, 45 at 09:59:08 | 172.20.7.223:50174 ↔ 20.190.177.84:443 | SYN, SYN-ACK, ACK handshake to Microsoft sign-in |
| **tls** | 2,474 | Frame 46 Client Hello at 09:59:08.312 | 172.20.7.223 → 20.190.177.84:443 | TLS 1.2, server name login.microsoftonline.com |

**ICMP (Internet Control Message Protocol)** carries diagnostic and error messages between devices. Ping uses echo request (type 8) and echo reply (type 0). It sits at the network layer, so it has no port numbers. The four replies arrived within a few milliseconds and the replies had a TTL of 59 against 128 on my requests. TTL (time to live) is a hop counter that falls by one at each router; a reply starting at 64 and arriving at 59 suggests roughly five hops, so the destination is not on the local network. **Why it matters:** attackers use ICMP sweeps to discover which hosts are alive, so many unexpected pings to different addresses would be a warning sign. Four requests to a destination I chose myself is not.

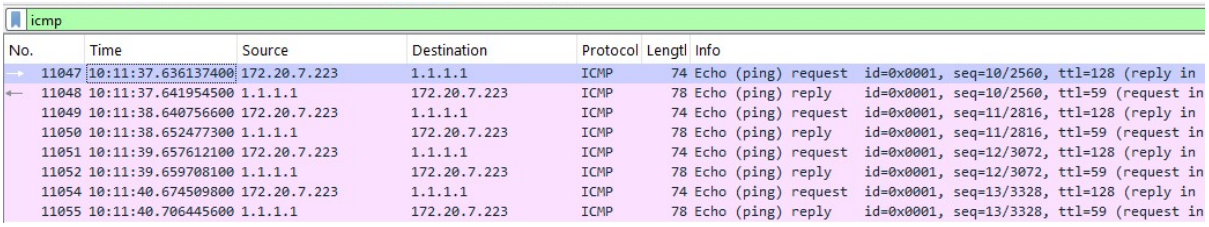



*Figure 3. ICMP filter: four echo requests (TTL 128) and four replies (TTL 59) between 172.20.7.223 and 1.1.1.1.*

**DNS (Domain Name System)** translates a name such as example.com into the IP address needed to connect. My computer sent a query from port 51747 to the DNS server 172.20.6.1 on UDP port 53, and the reply contained the same two IPv4 addresses that nslookup printed. The capture also holds the reverse lookup of the DNS server itself (which resolved to unifi.localdomain) and an AAAA query returning the IPv6 addresses. **Why it matters:** DNS happens before almost every connection and is usually unencrypted, so it reveals which names a computer tried to reach. Malware commonly uses DNS to find its control server, which makes DNS logs one of the earliest places to spot an infection.

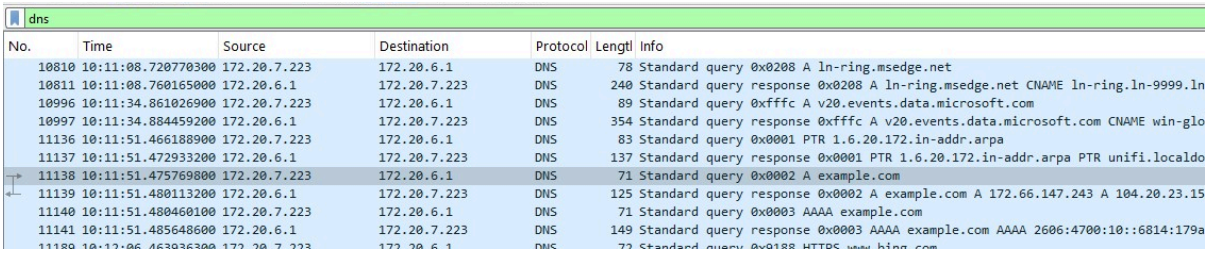



*Figure 4. DNS filter: query for example.com (frame 11138) and its response (frame 11139) from the local DNS server.*

**TCP (Transmission Control Protocol)** provides reliable, ordered delivery by first building a connection with a three-way handshake. In frames 40, 44 and 45 my computer sent SYN, the server answered SYN-ACK, and my computer confirmed with ACK, opening a connection to port 443 (HTTPS). Port 50174 on my side is a temporary (ephemeral) port chosen for this one connection. **Why it matters:** the handshake is the footprint of every connection attempt, so an unexpected SYN to an unusual destination or port is how many intrusions and scans first appear. This handshake was a background Microsoft sign-in connection, not one of my commands.

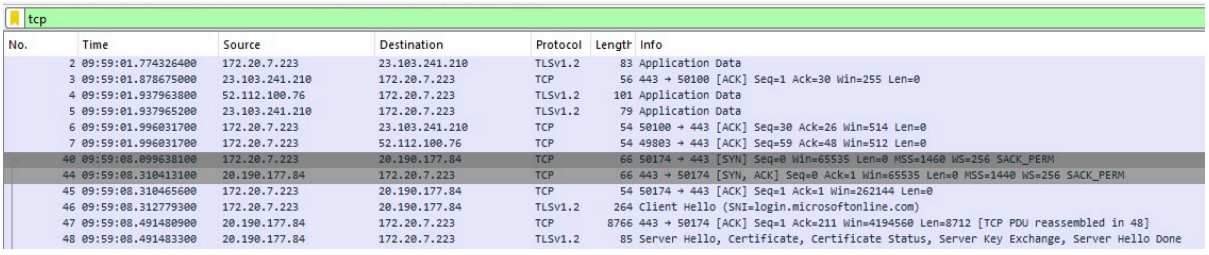
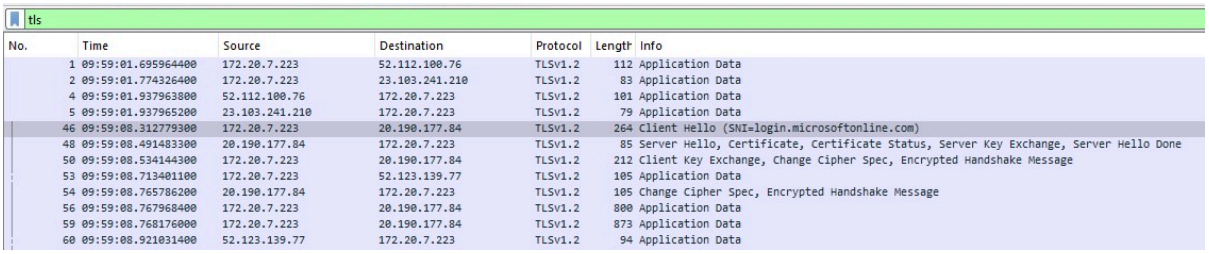



*Figure 5. TCP filter: handshake in frames 40, 44 and 45 to 20.190.177.84 on port 443.*

**TLS (Transport Layer Security)** encrypts data carried over TCP so that others on the network cannot read it. The client's first message (Client Hello, frame 46) names the server it wants, here login.microsoftonline.com, in readable text; after the handshake completes in frames 48 and 50 the content is encrypted. **Why it matters:** defenders cannot read encrypted content, so they depend on metadata such as the destination name, timing and volume. That metadata is exactly what lets an analyst judge whether a connection is expected.


tls

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 09:59:01.695964400 | 172.20.7.223 | 52.112.100.76 | TLSv1.2 | 112 | Application Data |
| 2 | 09:59:01.774326400 | 172.20.7.223 | 23.103.241.210 | TLSv1.2 | 83 | Application Data |
| 4 | 09:59:01.937963800 | 52.112.100.76 | 172.20.7.223 | TLSv1.2 | 101 | Application Data |
| 5 | 09:59:01.937965200 | 23.103.241.210 | 172.20.7.223 | TLSv1.2 | 79 | Application Data |
| 46 | 09:59:08.312779300 | 172.20.7.223 | 20.190.177.84 | TLSv1.2 | 264 | Client Hello (SNI=login.microsoftonline.com) |
| 48 | 09:59:08.491483300 | 20.190.177.84 | 172.20.7.223 | TLSv1.2 | 85 | Server Hello, Certificate, Certificate Status, Server Key Exchange, Server Hello Done |
| 50 | 09:59:08.534144300 | 172.20.7.223 | 20.190.177.84 | TLSv1.2 | 212 | Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message |
| 53 | 09:59:08.713401100 | 172.20.7.223 | 52.123.139.77 | TLSv1.2 | 105 | Application Data |
| 54 | 09:59:08.765786200 | 20.190.177.84 | 172.20.7.223 | TLSv1.2 | 105 | Change Cipher Spec, Encrypted Handshake Message |
| 56 | 09:59:08.767968400 | 172.20.7.223 | 20.190.177.84 | TLSv1.2 | 800 | Application Data |
| 59 | 09:59:08.768176000 | 172.20.7.223 | 20.190.177.84 | TLSv1.2 | 873 | Application Data |
| 60 | 09:59:08.921031400 | 52.123.139.77 | 172.20.7.223 | TLSv1.2 | 94 | Application Data |

*Figure 6. TLS filter: Client Hello naming login.microsoftonline.com, followed by the server's handshake and key exchange.*

**Reported limitation: no example.com connection captured**

A search of the capture found **no TCP or TLS packets to example.com's addresses**, even though DNS resolved the name. I did not confirm why. The capture does contain QUIC traffic on UDP port 443, which browsers can use instead of TCP, and a cached page is another possibility, but neither was verified. I used the genuine Microsoft handshake above as the TCP and TLS examples instead of inventing an example.com observation.

## 5.4 Windows Security log investigation (Day 2, Activity B)

The Security log held 24,863 events when opened; filtering on Event ID 4625 returned **547 failed logons**. Three were examined in detail:

| Time (16 Oct) | Logon type | Caller process | Status code | Source address |
|---|---|---|---|---|
| 1:22:46 AM | 2 | msedge.exe | 0xC000006E | - (none recorded) |
| 10:11:00 AM | 2 | msedge.exe | 0xC000006E | - (none recorded) |
| 12:01:21 PM | 2 | msedge.exe | 0xC000006E | - (none recorded) |

**Reading the fields. Logon type 2** is an interactive logon on the machine itself; the types that signal remote access are **3** (over the network) and **10** (Remote Desktop). **Source Network Address "-"** means Windows recorded no remote origin. The failure reason, "Unknown user name or bad password", is the text for status 0xC000006E. The logon process **Advapi** indicates an application asked Windows to check credentials through the Windows programming interface, rather than a person typing at the sign-in screen, which fits Edge's background credential checks. All three events share Logon ID 0x374A7, so they occurred in one logon session, and the account was the local account that was signed in. The Edge process ID changed between events, which is normal for a browser that runs many processes. Timestamps in the event XML are in UTC and 2 hours behind the displayed South African time, so the 10:11:00 AM event is 08:11:00Z in the XML.

**Why it matters:** a password-guessing attack looks different. It produces many 4625 events in quick succession, usually with logon type 3 or 10, a source IP address, and often several different account names. The three sampled events show none of these features, but three events cannot represent 547, so I searched the whole log with custom XML filters:

| # | Search | Events found | Log total at the time |
|---|---|---|---|
| 1 | Event 4625 with logon type 3 or 10 (network or Remote Desktop) | **0** | 24,905 |
| 2 | Event 4625 with a source IP address (not "-") | **0** | 24,819 |
| 3 | Event 4625 against any account other than the signed-in local account | **0** | 24,820 |
| 4 | Event 4624 (successful logon) with logon type 3 or 10 | **0** | 24,821 |

The successful logon examined (4624 at 12:04:02 PM) was **logon type 5**, a service logon by NT AUTHORITY\SYSTEM started by services.exe. This is Windows starting one of its own services and is not a person signing in. It sat beside an event 4672 (special privileges assigned), which is expected for the SYSTEM account. The log total changes between searches (24,819 to 24,905) because the Security log is a circular log that overwrites its oldest events as new ones arrive, which has an evidence-retention consequence discussed in Section 7.

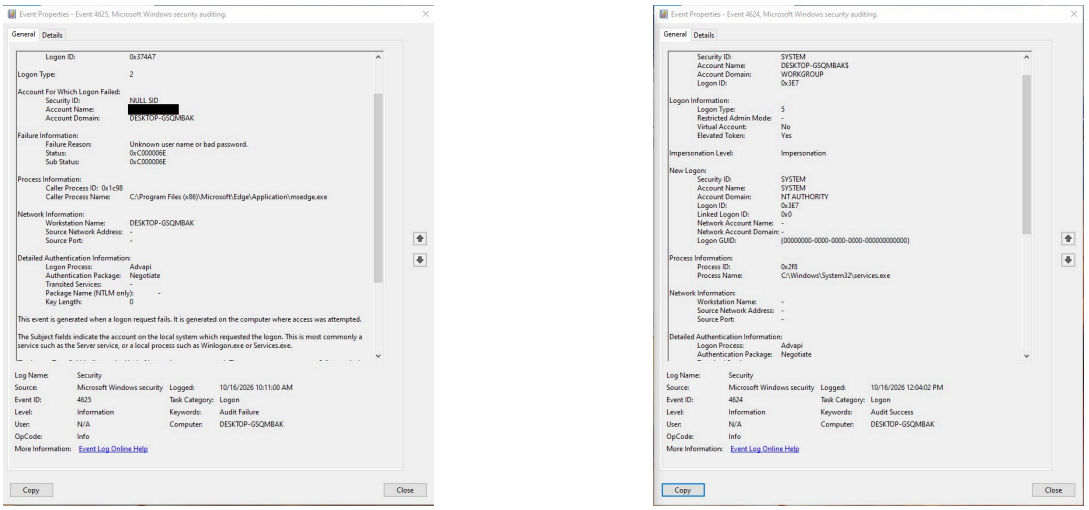


*Figure 7. (a) Event 4625: logon type 2, status 0xC000006E, caller msedge.exe, source address "-" (account name redacted). (b) Event 4624: logon type 5, SYSTEM service logon.*

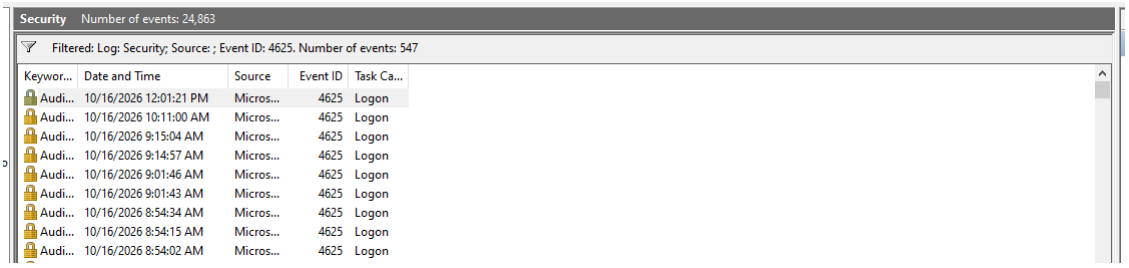



*Figure 8. Security log filtered to Event ID 4625: 547 events, newest first.*

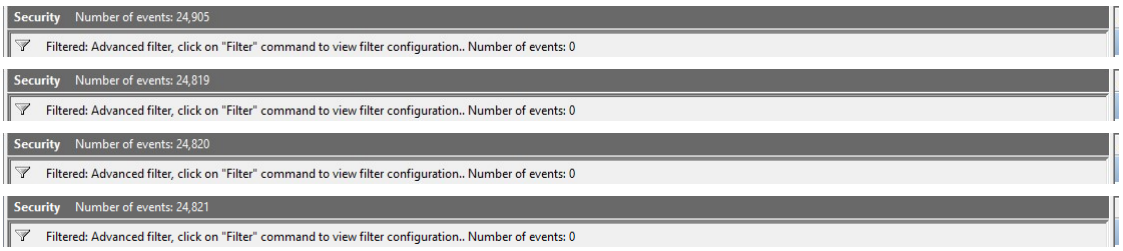



*Figure 9. Custom-filter searches 1 to 4 (top to bottom), each returning "Number of events: 0".*

### 5.5 Correlation and assessment (Day 2, Activity C)

Neither evidence source shows the signs a real intrusion would leave, and the two sources agree with each other:

| Indicator | What an attack would typically show | What was observed |
|---|---|---|
| Network destinations | Connections to unfamiliar servers, repeated beaconing, unexpected ports | Traffic matched my commands plus Microsoft sign-in and cloud services; no unexpected destination in the views examined |
| Failed logons | Bursts of 4625 with type 3 or 10, a source IP, many account names | All 547 were local, none had an IP address, none targeted another account |
| Successful remote access | 4624 with type 3 or 10 from an unfamiliar source | 0 events; the only 4624 examined was a SYSTEM service logon |
| Antivirus detection | Unplanned detections, or detections that failed to remove the file | One detection, the deliberate EICAR test, removed successfully |

**Conclusion: there is insufficient evidence that a genuine attack occurred.** The detection on Day 1 was the planned test, the network traffic was explained, and the Security log lacks the remote and guessing patterns of an attack. This is a conclusion about the evidence I held, not a guarantee. I opened only 3 of 547 failures and did not inspect all 14,323 packets, and the cause of the Edge failures is inferred. A zero-result search can also be caused by a mistake in the filter, so a control query (for example logon type 2, expecting about 547 results) should be run to prove the filters work.

## 6. Incident response plan

This plan covers the reported situation in Section 2. Each step states what to do, why, and what the lab showed. Evidence must be preserved before anything is cleaned.

## Step 1: Verify the alert

**Goal:** decide whether this is a true detection, a false positive or a test, and whether the file ever ran. **Why:** every alert should be treated as genuine until verified, otherwise real incidents are dismissed and false alarms waste effort.

- **Tell the user** not to open, forward, restore or delete the file, to leave the computer switched on, and to photograph or screenshot the warning. Switching it off destroys volatile evidence such as running processes and network connections.
- **Read Protection history** on the workstation and record the alert name, time, severity, file path, source and action. In the lab this gave Virus:DOS/EICAR_Test_File, Severe, Removed, with the source URL. “Removed” or “Quarantined” means the threat was contained; “Allowed” or a failed removal means it was not.
- **Establish whether it executed.** Check the process that wrote the file, running processes and recent logon events (4624, 4625, and 4688 if process-creation auditing is on). A file removed on write and never started is far lower risk than one that ran.
- **Check the email source.** Review the sender, headers and links, and search the mail system for the same attachment sent to other staff. A message that reached one inbox usually reached many.
- **Identify the file** by its SHA-256 hash (Get-FileHash in PowerShell) and look up the hash in a reputation service, so the file itself does not have to be uploaded.

## Step 2: Preserve evidence

**Goal:** keep a record that can support the investigation, a disciplinary process or legal action. **Why:** cleaning up first destroys the proof, and the Security log is circular (about 24,900 events here), so old entries are overwritten continuously.

- **Collect the most volatile data first:** running processes, active network connections and, if the network is suspect, a Wireshark capture limited to authorised equipment. Then collect logs and files.
- **Export the Security log** (Event Viewer, Save All Events As, .evtx) and the Protection history details, and keep the email as an .eml file with full headers.
- **Work on copies and record hashes.** Hash each evidence file when collected and again when copied, so any later change is detectable. Keep a **chain-of-custody** note of who collected what, when and where it is stored.
- **Record time zones.** Event Viewer displays local time but its XML uses UTC; mixing them is a common cause of wrong timelines.

## Step 3: Decide whether to isolate the workstation

**Why this is a decision and not a reflex:** isolation stops malware spreading or contacting its operator, but it also stops the user working and cuts off the live data needed to investigate. The choice depends on evidence:

| Keep connected, monitor closely | Isolate immediately |
|---|---|
| Defender blocked or removed the file when it was written<br>No sign the file executed<br>No unexpected traffic, no unfamiliar logons, no new accounts<br>**This matches the lab result** | The file executed, or Defender could not remove it<br>Unknown outbound connections, repeated beaconing, or remote logons (type 3 or 10) from unfamiliar sources<br>Signs of spread, mass file changes or credential theft, or a privileged user or sensitive data involved |

**How to isolate:** disconnect the Wi-Fi or Ethernet cable, or use the endpoint security platform’s isolate function if one exists, and keep the machine **powered on**. Escalate to the security lead or IT manager at the same time.

## Step 4: Remove confirmed threats

- **Let Defender remove the file**, update security intelligence, then run a **full scan** (the quick scan covered 31,415 files in 47 seconds, but a full scan checks everything). A **Microsoft Defender Offline scan** handles malware that hides while Windows is running.
- **Remove the cause from the organisation:** delete the message from every mailbox that received it, and block the sender, domain, URL and file hash.
- **If the file executed, assume credentials are exposed.** Reset the user’s passwords from a clean device, revoke active sessions and check for persistence (new startup items, scheduled tasks, services or accounts).
- **If cleaning cannot be trusted, rebuild.** Re-imaging from a known-good image is safer than hoping every trace was found.

## Step 5: Restore normal operations safely

- **Verify the machine is clean:** a clean full scan, no suspicious traffic, and real-time protection, firewall and updates all confirmed on.
- **Restore data from a backup taken before the incident** and check the backup itself is clean. Reconnect to the network in stages, and watch the machine more closely for a period afterwards (for example one to two weeks).
- **Close the loop:** brief the user, record the incident, and hold a review (Section 8). If personal information may have been accessed, legal duties to notify the regulator and affected people (for example under POPIA in South Africa) must be assessed.

## 7. Recommendations

Recommendations are ordered by priority. High items address gaps that would slow or hide the response to a real incident.

| Priority | Control | Action | Why (finding) |
|---|---|---|---|
| **High** F-01 | **Keep and lock down Defender** | Keep real-time and cloud-delivered protection and sample submission on, enforce them centrally and enable Tamper Protection. | Detection worked only because real-time protection was on. Malware often tries to switch protection off. |
| **High** F-03 | **Centralise alerts** | Forward Defender alerts and Security logs to a central console or SIEM and notify IT automatically. | Protection history is stored per machine and the pop-up was missed. Unseen alerts are not acted on. |
| **High** F-02 | **Filter email attachments** | Add attachment scanning or sandboxing, block risky file types and mark external senders. | Detection happens once the file is on disk. Stopping it earlier removes the risk. |
| **Medium** F-05 | **Protect log evidence** | Increase Security log size and forward copies off the machine. | Events are being overwritten, so evidence of an older incident could be lost. |
| **Medium** F-04 | **Investigate the Edge failures** | Find why Edge generates failed logons about every four hours (around :22:4x), including overnight. | Harmless is only an assumption. An unexplained recurring pattern should be explained. |
| **Medium** F-06 | **Use the right network profile** | Set the Public profile on shared or untrusted Wi-Fi and confirm which profile applied on Day 2. | Private is more permissive and suits trusted networks only. |
| **Medium** All | **Train staff, define a reporting path** | Teach staff to stop, not open or forward, and report at once, with a simple reporting route. | The user's first reaction decides whether evidence survives (Section 6). |
| **Medium** All | **Resilience basics** | Offline backups, patching, least privilege and multi-factor authentication. | Limits the damage if an attachment ever does execute. |
| **Low** F-07 | **Test regularly** | Repeat the EICAR test periodically and validate log searches with a control query. | Controls drift; a zero-result filter is only trustworthy once proven (Section 5.5). |

## 8. Lessons learned

**What worked.** Starting with a baseline made the detection meaningful. Combining network and host evidence gave two independent views that agreed. Searching the whole Security log with filters, not just reading three events, turned an impression into a finding.

**What I would improve.**

- **Capture discipline.** I left Wireshark running about 18 minutes, which added many packets of background traffic, and I did not record when I visited example.com. A shorter capture, noting start and end times, would give cleaner evidence.
- **Unresolved questions.** Why no example.com TCP or TLS traffic was captured, and why Edge produces the failed logons, remain unconfirmed.
- **Coverage.** Only 3 of 547 failed logons were opened individually, the logs cover only the retained period, and I ran no control query to prove the zero-result filters.

**Overall.** The organisation's endpoint protection did its job, and the investigation process produced a reasoned, evidence-based conclusion with its limits stated. The weaknesses found were about visibility and retention (alerts seen only locally, logs overwritten), not about detection itself.